

Portfolio · Computer Vision / ML Engineering

DFU Foot Analysis Pipeline

Shared DINOv3 backbone 위에서 발·궤양 세그멘테이션과 당뇨발(DFU) 이진 분류를 통합한 의료 영상 분석 파이프라인

2026.06 — 2026.07

Solo Project · 100%

PyTorch · DINOv3 · Gradio

프로젝트 설명

작업 기간

약 4주

2026.06.12 — 2026.07.07 · 설계 → 학습 → 추론 UI까지 End-to-End

기여도 / 역할

100% · Owner

아키텍처 설계, 데이터 파이프라인, 학습/평가, 추론 게이트,
Gradio+WebRTC UI

개요

단일 입력 이미지에서 발 영역 탐지 → 궤양 세그멘테이션 → **DFU/Other** 분류를 수행하는 통합 파이프라인. frozen DINOv3 ViT-S/16
을 공유하고 task별 head만 독립 학습·조합하여 학습 효율과 추론 속도를 동시에 확보했습니다.

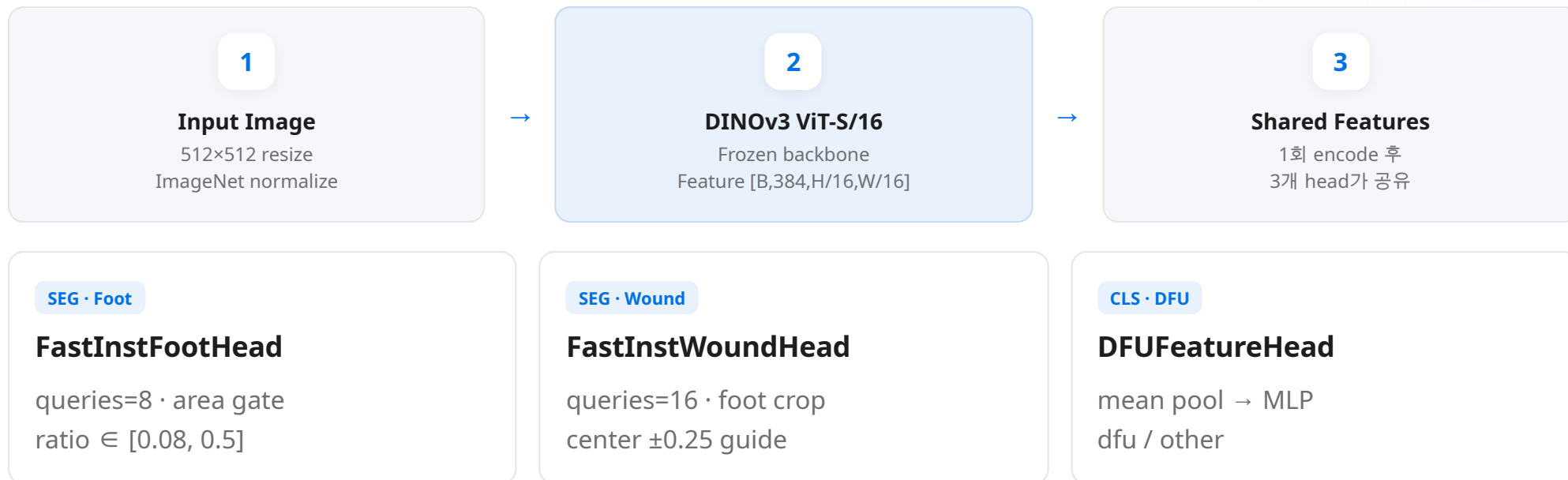
Foot Dice 0.959

Wound Dice 0.814

DFU Test Acc 96.2%

DFU Test F1 93.1%

전체 아키텍처



학습: `train.py --task {foot|wound|dfu}` · 추론: `DFUPipelineModel`이 checkpoint 조합

스킬 정리

COMPUTER VISION / DEEP LEARNING

Semantic Segmentation

Image Classification

Vision Transformer

Transfer Learning

Multi-task Heads

MLOPS / EXPERIMENT

Checkpoint Design

Early Stopping

AMP Training

Hold-out Evaluation

Ablation Study

DATA ENGINEERING

COCO / ImageFolder

Hard Negative Mining

Foot Crop Pipeline

Class Imbalance

Multi-source Merge

PRODUCTIZATION

Inference Gating

Gradio UI

WebRTC Realtime

Capture Guidance

CLI Tooling

사용 기술 스택

CORE ML

PyTorch · Torchvision

학습/추론 루프, AMP, frozen
backbone + light head 최적화

BACKBONE

DINOv3 ViT-S/16

HuggingFace Transformers 로컬 스
냅샷 · 384-dim feature map

HEADS

FastInst-style · MLP

query-based binary mask · spatial
mean pool classifier

SERVING / UI

Gradio + FastRTC

이미지 탭 + WebRTC 실시간 오버레이
(0.5s JSON 갱신)

DATA

COCO · ImageFolder

Roboflow, FUSeg, AI Hub, Kaggle
DFU, 내부 PART A

INFRA

CUDA · Python 3.10+

환경 변수 기반 경로 · verify_setup ·
evaluate scripts

Shared Backbone · 1회 Encode

추론 시 backbone을 한 번만 실행하고, 동일 feature map을 foot / wound / dfu head가 공유합니다.

왜 중요한가

ViT backbone이 연산의 대부분을 차지하므로, multi-task에서도 latency를 최소화하면서 head를 독립적으로 교체·재학습할 수 있습니다.

backbone 1×

head 독립 checkpoint

task별 재학습

```
# models/pipeline_model.py
class DFUPipelineModel(nn.Module):
    def encode(self, x):
        return self.backbone(x) # 1회

    def predict_foot_logits(self, features, ...):
        return self.foot_head(features)

    def predict_wound_logits(self, features, ...):
        return self.wound_head(features)

    def predict_dfu_logits(self, features):
        return self.dfu_head(features)
```

Gate Logic + Foot Feature Crop

INFERENCE GATE

Foot: area ratio $\in [0.08, 0.5]$

Wound: foot 탐지 + 중앙 ± 0.25

DFU: foot 탐지 시 실행

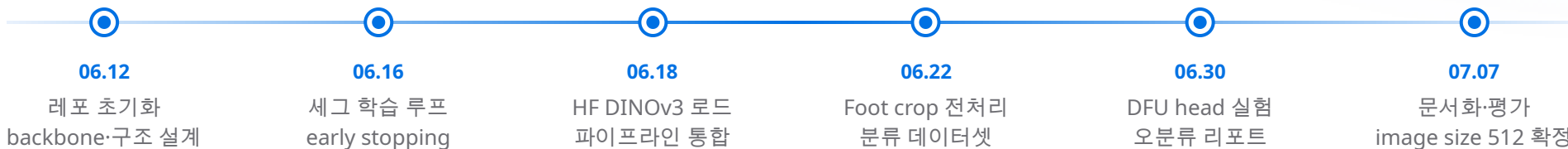
WOUND CROP

foot mask bbox를 margin(0.15)만큼 확장한 feature crop만
wound head에 전달 → 배경 노이즈 감소, 국소 정밀도 향상

```
# inference/pipeline.py (요약)
def predict_wound_logits_for_foot_crop(...):
    bbox = bbox_from_mask(foot_mask, margin)
    crop = features[..., y0:y1, x0:x1]
    logits = model.predict_wound_logits(crop, size)
    full = fill_neg_inf(model_size)
    full[..., y0:y1, x0:x1] = logits
    return full

# DFU head: spatial mean pooling
pooled = features.mean(dim=(2, 3))
return mlp(pooled) # dfu / other
```

프로젝트 진행 타임라인



3

독립 Task Heads

1×

Backbone Encode / Frame

E2E

Train → Infer → UI

발 세그멘테이션

P

문제

다양한 촬영 각도·배경에서 발 영역만 안정적으로 분리해야 하며, 신체 오탐을 줄여야 함

E

실행

Roboflow foot + hard negative(body) · FastInst
queries=8 · neg oversample·loss weight

R

성과

Val Dice 0.959
Val IoU 0.935
(foot_head_v2, 512)

I

인사이트

Hard negative와 empty-mask 가중치가 오탐 억제에 결정적. 이후 분류 crop의 품질 기반이 됨

Run	Image Size	Best Val Dice	Best Val IoU
foot_head_v1	512	0.954	0.933
foot_head_v2	512	0.959	0.935

궤양 세그멘테이션

P

문제

궤양 영역이 작고 경계가 모호하며,
전체 이미지 기준 학습 시 배경에 민감

E

실행

FUSeg + Wound Image Dataset ·
queries=16 · inference 시 foot
bbox feature crop

R

성과

Val Dice 0.814
Val IoU 0.738
(wound_head_v1)

I

인사이트

학습은 full image, 추론은 foot
crop으로 ROI를 좁히면 실사용 정밀
도가 올라감. Gate로 잘못된 촬영을
사전 차단

데이터

FUSeg + WID

positive / negative 혼합

CROP MARGIN

0.15

infer.py 기본값

GUIDE GATE

±0.25

화면 중앙 정렬 조건

당뇨발 이진 분류

P

문제

출처·구도가 다른 데이터에서 dfu vs other를 일반화해야 하며, 클래스 불균형 존재

E

실행

AI Hub + Kaggle + PART A · foot crop 전처리 · Linear/MLP ablation · F1 기준 best

R

성과

Test Acc 96.2%
Test F1 93.1%
(mlp + aug)

I

인사이트

Foot crop이 도메인 갭을 줄임.
MLP+aug가 Linear 대비 F1 개선.
오분류 리포트로 실패 패턴 추적

Run	Head	Val Acc	Val F1	Test Acc	Test F1
dfu_head_v1	linear	95.1%	90.9%	—	—
dfu_head_mlp_1e3	mlp	98.0%	93.4%	96.1%	92.9%
dfu_head_mlp_1e3_aug	mlp+aug	97.8%	94.4%	96.2%	93.1%

— RESULTS

기능 · 성능 테스트 요약

0.959

Foot Val Dice

0.814

Wound Val Dice

96.2%

DFU Test Accuracy

93.1%

DFU Test F1

기능	검증 방법	결과 / 지표
통합 Inference CLI	<code>infer.py</code> · <code>mask/overlay/JSON</code>	foot → wound → dfu 게이트 동작 확인
Hold-out Evaluation	<code>scripts/evaluate.py</code>	<code>dice/iou</code> · <code>acc/precision/recall/f1</code>
DFU Head 비교	<code>evaluate_dfu_heads_on_test.py</code>	MLP+aug가 Test F1 best
Realtime UI	Gradio + FastRTC	오버레이 + 0.5s JSON 갱신
촬영 가이드	area / center gate	잘못된 입력 사전 차단 · UX 메시지

GitHub · 산출물

SOURCE REPOSITORY

yoonsula / dfu-project

발·궤양 세그멘테이션 + DFU 분류 통합 파이프라인

github.com/yoonsula/dfu-project

TRAINING

train.py

task dispatcher · foot / wound / dfu
독립 학습

INFERENCE

infer.py

gated pipeline · timing JSON ·
overlay export

DEMO UI

app_gradio.py

이미지 분석 + WebRTC 실시간 탭

하나의 **frozen backbone**으로
발 탐지 · 궤양 분할 · DFU 분류를 묶고,
게이트와 crop으로 **실사용 정밀도**까지 끌어올린
의료 영상 분석 파이프라인.

Dice 0.959 / 0.814

DFU F1 93.1%

github.com/yoonsula/dfu-project